

2303A51813

J MUKHESH

```
1. import java.util.*;

public class segment{

    static int[] arr;

    static int[] tree;

    static int n;


    static int gcd(int a,int b){

        while(a != 0){

            int c = b % a;

            b = a;

            a = c;

        }

        return b;

    }

    static void build(int idx,int str,int end){

        if(str == end){

            tree[idx] = arr[str];

            return ;

        }

        int mid = (str + end)/2;

        build(2 * idx + 1,str,mid);

        build(2 * idx + 2,mid + 1,end);

        tree[idx] = gcd(tree[2 * idx + 1],tree[2 * idx + 2]);

    }


    static void update(int idx,int str,int end,int x,int val){
```

```

    if(str == end){
        arr[x] = val;
        tree[idx] = val;
        return ;
    }
    int mid = (str + end)/2;
    if(x <= mid)
        update(2 * idx + 1, str, mid, x, val);
    else
        update(2 * idx + 2, mid + 1, end, x, val);
    tree[idx] = gcd(tree[2 * idx + 1], tree[2 * idx + 2]);
}

static int query(int idx, int str, int end, int left, int right){
    if(left > end || right < str)
        return 0;
    if(left <= str && end <= right)
        return tree[idx];
    int mid = (str + end)/2;
    return gcd(query(2 * idx + 1, str, mid, left, right), query(2 * idx + 2, mid +
1, end, left, right));
}

public static void main(String args[]){
    Scanner s = new Scanner(System.in);
    n = s.nextInt();
    arr = new int[n];
    tree = new int[4 * n];

```

```

for(int i = 0; i < n; i++){
    arr[i] = s.nextInt();
}
int k = s.nextInt();
build(0,0,n - 1);
for(int i = 0; i < k; i++){
    int op = s.nextInt();
    if(op == 1)
        update(0,0,n - 1,s.nextInt(),s.nextInt());
    else if(op == 2)
        System.out.print(query(0,0,n - 1,s.nextInt(),s.nextInt()));
    else
        System.out.print("Enter a valid number");
}
}
}

```

---

2. static void build(int idx,int str,int end){

```

    if(str == end){
        tree[idx] = arr[str];
        return ;
    }
    int mid = (str + end)/2;
    build(2 * idx + 1,str,mid);
    build(2 * idx + 2,mid + 1,end);
    tree[idx] = tree[2 * idx + 1] * tree[2 * idx + 2];
}

```

```

static void update(int idx,int str,int end,int x,int val){

```

```

    if(str == end){
        arr[x] = val;
        tree[idx] = val;
        return ;
    }
    int mid = (str + end)/2;
    if(x <= mid)
        update(2 * idx + 1, str, mid, x, val);
    else
        update(2 * idx + 2, mid + 1, end, x, val);
    tree[idx] = tree[2 * idx + 1] * tree[2 * idx + 2];
}

static int query(int idx, int str, int end, int left, int right){
    if(left > end || right < str)
        return 1;
    if(left <= str && end <= right)
        return tree[idx];
    int mid = (str + end)/2;
    return query(2 * idx + 1, str, mid, left, right) * query(2 * idx + 2, mid + 1, end, left, right);
}

```

---

3. static void build(int idx, int str, int end){

```

    if(str == end){
        tree[idx] = arr[str];
        return ;
    }
    int mid = (str + end)/2;
    build(2 * idx + 1, str, mid);

```

```

    build(2 * idx + 2, mid + 1, end);

    tree[idx] = tree[2 * idx + 1] | tree[2 * idx + 2];
}

static void update(int idx, int str, int end, int x, int val){
    if(str == end){
        arr[x] = val;
        tree[idx] = val;
        return ;
    }
    int mid = (str + end)/2;
    if(x <= mid)
        update(2 * idx + 1, str, mid, x, val);
    else
        update(2 * idx + 2, mid + 1, end, x, val);
    tree[idx] = tree[2 * idx + 1] | tree[2 * idx + 2];
}

```

```

static int query(int idx, int str, int end, int left, int right){
    if(left > end || right < str)
        return 0;
    if(left <= str && end <= right)
        return tree[idx];
    int mid = (str + end)/2;
    return query(2 * idx + 1, str, mid, left, right) | query(2 * idx + 2, mid + 1, end, left, right);
}

```

---

```

4. static void build(int idx, int str, int end){
    if(str == end){

```

```

        tree[idx] = arr[str];
        return ;
    }
    int mid = (str + end)/2;
    build(2 * idx + 1,str,mid);
    build(2 * idx + 2,mid + 1,end);
    tree[idx] = tree[2 * idx + 1] + tree[2 * idx + 2];
}

```

```

static void update(int idx,int str,int end,int x,int val){
    if(str == end){
        arr[x] = val;
        tree[idx] = val;
        return ;
    }
    int mid = (str + end)/2;
    if(x <= mid)
        update(2 * idx + 1,str,mid,x,val);
    else
        update(2 * idx + 2,mid + 1,end,x,val);
    tree[idx] = tree[2 * idx + 1] + tree[2 * idx + 2];
}

```

```

static int query(int idx,int str,int end,int x){
    if(str == end)
        return str;
    int mid = (str + end)/2;
    int left = tree[2 * idx + 1];

```

```

    if(x <= left)

        return query(2 * idx + 1,str,mid,x);

    x = x - left;

    return query(2 * idx + 2,mid + 1,end,x);

}

```

---

```

6 static void update(int idx,int str,int end,int x,int val){

```

```

    if(str == end){

        arr[x] = val;

        tree[idx] = val;

        return ;

    }

    int mid = (str + end)/2;

    if(x <= mid)

        update(2 * idx + 1,str,mid,x,val);

    else

        update(2 * idx + 2,mid + 1,end,x,val);

    tree[idx] = tree[2 * idx + 1] + tree[2 * idx + 2];

}

```

```

static int query(int idx,int str,int end,int left,int right){

    if(left > end || right < str)

        return 0;

    if(left <= str && end <= right)

        return tree[idx];

    int mid = (str + end)/2;

    return query(2 * idx + 1,str,mid,left,right) + query(2 * idx + 2,mid + 1,end,left,right);

}

```

```

// update

```

```
if(op == 1){  
    int x = s.nextInt();  
    int val = s.nextInt();  
    if(val == 0)  
        val = 1;  
    else  
        val = 0;  
    update(0,0,n - 1,x,val);  
}
```

---